Technical Appendix (Notebook)

Group member: **Ziang Yang (zy234) Xianghao Zheng (xz473) Yanhe Zhu (yz1054) Yuyao Tang (yt237)**
Project: **Cubazoid Solver**
Repository root: `final project`
Appendix source outline: `docs/technical_appendix_outline.md`

---

1) Repository Snapshot & Environment Lock

This chapter freezes the exact repository state and runtime environment used for reported results.
Goal: a grader should be able to recreate the same environment and rerun commands from this chapter alone.

1.1 Snapshot Record (Frozen Baseline)

Validation timestamp (local): **2026-04-19 19:08:32 -04:00**
Timezone: **America/New_York**
Branch: **`main`**
Commit hash: **`10cdca32ec84ccf1f19ed75ae6ca76816322ef65`**
HEAD commit date: **2026-04-19 06:28:20 -0400**
HEAD commit subject: **`tex`**
Working tree status at snapshot: **clean**
Python version: **3.10.5**

Interpretation rule:

- All runtime numbers and status labels in this appendix should be interpreted as belonging to this snapshot unless a later chapter explicitly declares a new snapshot.

1.2 Repository File Map (Reproduction-Critical Files)

Path	Role	Why it matters
`cubazoid_solver.py`	CLI entry	Defines grader-facing command-line workflow (`--all` , `--case` , `--backend` , `--timeout-sec` , `--no-show`)
`cubazoid/__init__.py`	Public exports	Central import surface used by scripts/notebooks
`cubazoid/types.py`	Data contracts	Defines `Coord` , `Placement` , `PlacementOption`
`cubazoid/geometry.py`	Geometry preprocessing	Tensor-to-coordinates conversion, normalization, 24 rotations, unique orientations
`cubazoid/examples.py`	Canonical cases	Named benchmark cases, large-search cases, invalid-input cases
`cubazoid/solver.py`	Core logic	Input validation, MRV backtracking backend, exact-cover DLX backend, timeout handling
`cubazoid/api.py`	Backend wrapper	Unified `solve_and_visualize(...)` entry used by CLI/notebook
`cubazoid/visualization.py`	Rendering	3D voxel animation and GIF export
`demo_cubazoid_repro.ipynb`	Experiment runner	Batch execution, verification checks, JSON/CSV export
`outputs/all_case_results.json`	Structured results	Full machine-readable benchmark status + checks
`outputs/all_case_results.csv`	Tabular results	Human-readable benchmark table
`requirements.txt`	Environment lock file	Exact package pins for this appendix

Pipeline summary:

1. Cases are generated from `examples.py` .
2. Shapes are normalized/oriented through `geometry.py` .
3. Solver backend (`mrv` or `exact`) runs in `solver.py` .
4. Optional visualization is produced by `visualization.py` .
5. Batch outputs are exported to `outputs/*.json` and `outputs/*.csv` .

1.3 Environment Lock Strategy

Lock strategy used in this appendix:

- Primary environment manager: **Conda**
- Primary lock artifact: **`environment.yml`** (repo root)
- Secondary pin list: `requirements.txt` (referenced by `environment.yml` via pip section)
- Interpreter target: **Python 3.10.x** (validated on 3.10.5)
- Timing policy: use `--no-show` when collecting runtime results

Current lock files:

- `environment.yml`
- `requirements.txt`

Pinned dependency set (via `requirements.txt`):

```
numpy==1.24.2
matplotlib==3.5.2
Pillow==9.2.0
pandas==2.0.0
ipython==8.12.0
notebook==6.4.12
jupyterlab==3.4.8
ipykernel==6.22.0
```

Why this lock is sufficient for current project scope:

- Solver path depends mainly on `numpy`.
- Visualization path depends on `matplotlib + Pillow`.
- Notebook/reporting path depends on `pandas + jupyter stack`.

1.4 Exact Environment Setup Commands (Windows PowerShell, Conda)

```
# 0) Enter repo root
Set-Location "c:\Users\13681\Desktop\duke\ECE\PML\final project"

# 1) Create environment from lock file
conda env create -f environment.yml

# 2) Activate
conda activate cubazoid-repro

# 3) Verify interpreter and core packages
python --version
conda list numpy matplotlib pillow pandas ipython notebook jupyterlab ipykernel
```

If `conda activate` fails in PowerShell, initialize once and reopen shell:

```
conda init powershell
```

1.5 Smoke-Test Commands (Post-Install)

These commands validate that the locked environment can execute the project end-to-end.

```
# List all available cases (including large search cases)
python cubazoid_solver.py --list-cases --include-large

# Minimal solvable case, no GUI, MRV backend
python cubazoid_solver.py --case easy_2x2x2_two_4blocks --backend mrv --no-show --timeout-sec 120

# Optional exact backend sanity check on same small case
python cubazoid_solver.py --case easy_2x2x2_two_4blocks --backend exact --no-show --timeout-sec 120
```

Expected behavior:

- `--list-cases` prints all predefined names.
- Small case returns `Solved[mrv]` and `Solved[exact]`.
- No visualization window appears when `--no-show` is set.

1.6 Reproducibility and Audit Trail (Recommended Files)

For grading transparency, keep these artifacts after each benchmark run:

- `outputs/all_case_results.json`
- `outputs/all_case_results.csv`
- `outputs/all_case_gifs/*.gif` (for solved cases)

Recommended environment provenance export (Conda-based):

```
New-Item -ItemType Directory -Force outputs\env | Out-Null
conda env export -n cubazoid-repro | Out-File -Encoding utf8 outputs\env\conda_env_export_2026-04-19.yml
conda list -n cubazoid-repro | Out-File -Encoding utf8 outputs\env\conda_list_2026-04-19.txt
python --version | Out-File -Encoding utf8 outputs\env\python_version_2026-04-19.txt
git rev-parse HEAD | Out-File -Encoding utf8 outputs\env\commit_2026-04-19.txt
git status --short | Out-File -Encoding utf8 outputs\env\git_status_2026-04-19.txt
```

Recommended interpretation policy:

- If commit hash differs, treat results as a new experiment version.
- If dependency versions differ, rerun smoke tests before trusting timing comparisons.

- If working tree is not clean, report that condition explicitly.

1.7 File Integrity Anchors (Optional but Strongly Recommended)

For strict reproducibility, record cryptographic hashes of key files in the report appendix or artifact bundle.

Known SHA256 values at this chapter snapshot:

- `requirements.txt` : `cc9a1e82827d88266642d4949a4f36f9db73095f16d67a1c703cf1ef30faeecb`
- `cubazoid_solver.py` : `a964d2b57aea24fe428c1a538e8fdd9d6324d34ae8dc5c7a5d67625c5374c915`
- `cubazoid/solver.py` : `3cec3f4be444ad668c21eb8397a2395e9129662f07b2f45baaa5ca19dc5deb9b`
- `demo_cubazoid_repro.ipynb` : `acee5f7e44057219269329a6ef37f0ac0f520e022cc76eeabb956669e94f494f`

How to regenerate hashes later (PowerShell):

```
Get-FileHash requirements.txt -Algorithm SHA256
Get-FileHash cubazoid_solver.py -Algorithm SHA256
Get-FileHash cubazoid\solver.py -Algorithm SHA256
Get-FileHash demo_cubazoid_repro.ipynb -Algorithm SHA256
```

1.8 Validation Date and Scope Statement

Chapter 1 was validated on **2026-04-19** and covers:

- repository identity lock,
- environment dependency lock,
- installation and smoke-test commands,
- artifact and provenance policy.

This closes Chapter 1 (`Repository Snapshot & Environment Lock`). Later chapters can now focus on algorithm internals, experiments, correctness checks, ablations, and demo behavior.

---

2) Problem Contract

This chapter defines the formal behavior contract of the solver system: what input is accepted, what output is produced, what is rejected, and how timeout is interpreted.

Contract scope in this chapter:

- Input format (list of 3D binary tensors)
- Output format (placement list or `None`)
- Validation rules (`[3,5]` piece size, connectivity, perfect-cube volume, timeout)
- Reporting semantics for invalid/unsolved/timed-out outcomes

2.1 Function-Level Contract (External View)

Primary runtime contract used by CLI and notebook:

```
solve_and_visualize(
    shape_tensors: List[np.ndarray],
    save_path: Optional[str] = None,
    interval: int = 900,
    show: bool = True,
    backend: str = "mrv",
    timeout_sec: Optional[float] = None,
) -> Optional[List[Placement]]
```

Backend solver contract:

```
solver.solve(max_seconds: Optional[float] = None) -> Optional[List[Placement]]
```

Interpretation:

- Return type is always either
  1. `List[Placement]` (valid cube-forming configuration), or
  2. `None` (invalid input / no solution found / timed out).

Important implication:

- `None` is intentionally overloaded at the solver return boundary.
- Distinguishing *why* `None` happened requires checking metadata fields (`infeasible_reason` , `timed_out`) in the caller/report pipeline.

2.2 Input Format Contract

2.2.1 Canonical Input Type

Input is a Python list where each element is one piece tensor:

```
shape_tensors: List[np.ndarray]
```

Each piece tensor must satisfy:

- Tensor is 3-dimensional (`ndim == 3`)
- Occupied cells are values `> 0` (treated as filled voxels)
- Empty cells are `0`
- At least one occupied voxel must exist

Internally, each tensor is converted to normalized coordinate tuples by:

1. `np.argwhere(arr > 0)` to extract voxel coordinates,
2. shifting coordinates so minimum x/y/z becomes 0,
3. sorting coordinates to create a canonical piece representation.

2.2.2 What "3D Binary Tensor" Means in Practice

Although source tensors are usually `0/1`, extraction logic uses `arr > 0`. Therefore:

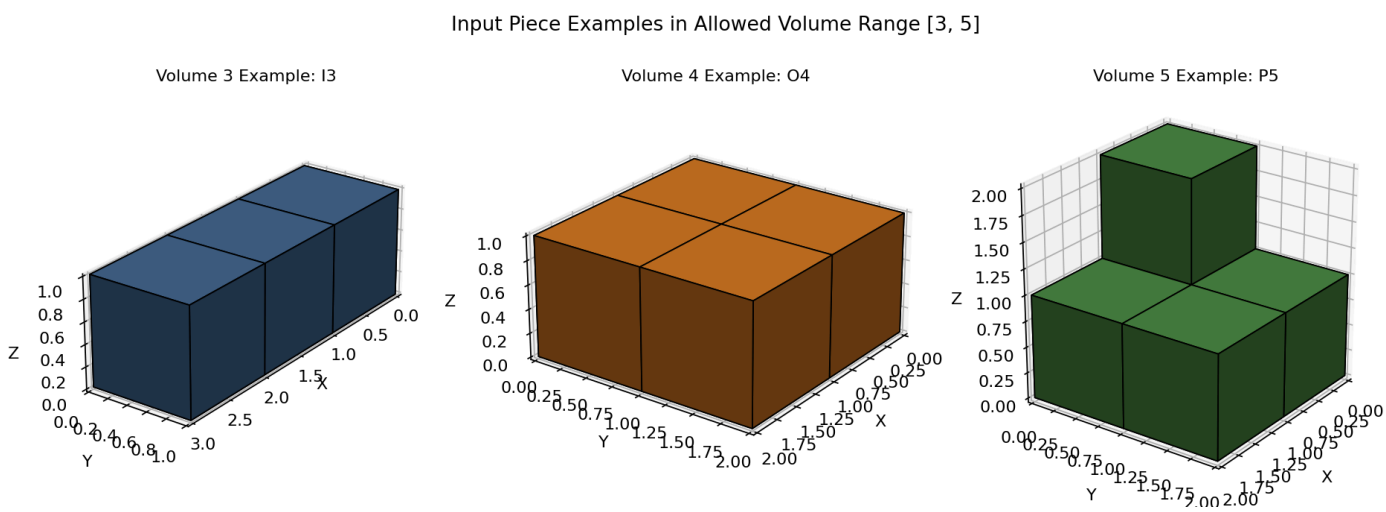
- any positive numeric value is interpreted as occupied,
- non-positive values are treated as empty.

For reproducible clarity, all project-generated pieces use explicit binary tensors (`uint8` , values 0 or 1).

2.3 Visual Intuition for Allowed Piece Volumes (3, 4, 5)

Below are one concrete example piece for each allowed volume class in this project contract.

- Volume 3 example: `I3`
- Volume 4 example: `O4`
- Volume 5 example: `P5`



Why this matters for the contract:

- The solver accepts only pieces with voxel count in `[3, 5]` .
- The figure provides a direct geometric intuition for that accepted range.

2.4 Output Format Contract

Returned solution object:

```
Optional[List[Placement]]
```

Where each `Placement` record contains:

```
Placement(
    piece_id: int,
    orientation_id: int,
    cells: Tuple[Coord, ...],
)

Coord = Tuple[int, int, int]
```

Field semantics:

- `piece_id` : index of the original input piece in `shape_tensors`

- `orientation_id` : orientation index within that piece's unique orientation list
- `cells` : absolute cube coordinates occupied by that placed piece in the target `n x n x n` cube

A returned placement list is valid only if all are true:

1. Each input piece appears exactly once (`piece_id` forms a permutation of `0..k-1`)
2. No two placements overlap in occupied cells
3. Union of all occupied cells equals the full cube volume (`n^3` cells)

If these cannot be achieved, return is `None` .

2.5 Validation Rules Implemented in Code

The following rules are enforced before/during search:

Rule A: Piece size must be in `[3, 5]`

- For each piece, voxel count is computed.
- Any piece with volume `<3` or `>5` marks the case infeasible.
- Infeasibility reason is stored in `infeasible_reason` .

Rule B: Piece must be connected (6-neighbor)

- Connectivity uses BFS/queue on voxel coordinates.
- Adjacency is axis-aligned only (+/-x, +/-y, +/-z), not diagonal.
- Default behavior is strict rejection of disconnected pieces.

Rule C: Total volume must be a perfect cube

- Let total occupied voxels be `V` .
- Solver computes `n = round(V ** (1/3))` and requires `n^3 == V` .
- If not perfect cube, case is rejected before search.

Rule D: Timeout is per case

- Each `solve(...)` call may receive `max_seconds` .
- `max_seconds <= 0` or `None` means timeout disabled.
- On timeout hit, solver sets `timed_out = True` and returns `None` .

Additional practical pre-check (important behavior detail):

- If any piece has zero legal placements even on an empty cube (`unplaceable_on_empty`), the case returns `None` immediately.
- This is not a separate user-facing contract item in the outline, but it is a real implemented rejection path.

2.6 Status Interpretation at Report Layer

Because solver return is `Optional[List[Placement]]` , reporting logic maps outcomes into explicit status labels.

Typical mapping used in this project notebook:

1. `timed_out == True` -> `status = "timeout"`
2. else if `infeasible_reason` not empty -> `status = "invalid_input"`
3. else if `placements is None` -> `status = "unsolved"`
4. else -> `status = "solved"`

Contract-level conclusion:

- `None` at raw API layer does **not** alone distinguish invalid input, search failure, or timeout.
- A reproducible technical report must publish these status rules explicitly (done here).

2.7 Input/Output Examples (Spec-Oriented)

Example 1: Valid input form

```
# schematic only
shape_tensors = [piece0_tensor, piece1_tensor, ..., pieceK_tensor]  # each is 3D np.ndarray
placements = solver.solve(max_seconds=120)
# placements is either List[Placement] or None
```

Example 2: Invalid total volume

```
Total voxel count = 24
24 is not a perfect cube
-> infeasible_reason set
-> solve(...) returns None
```

Example 3: Timeout-limited run

```
Given max_seconds = 120
Search exceeds 120s
-> timed_out = True
-> solve(...) returns None
-> report status should be timeout (not generic unsolved)
```

2.8 Chapter 2 Closeout

This chapter fixed the problem contract used by all later sections:

- exactly what input shape representation is accepted,
- exactly what output object means,
- exactly which validation rules are enforced,
- and exactly how timeout and `None` are interpreted in reporting.

With this contract frozen, the next chapter can focus on solver internals without ambiguity about interface behavior.

---

3) Solver Design

This chapter is the algorithmic core of this project.
For each backend (`mrv` , `exact`), I provide:

1. pseudocode of the full control flow,
2. **verbatim source code snippets from this repository**,
3. direct explanation tied to those snippets.

Primary code files:

- `cubazoid/geometry.py`
- `cubazoid/solver.py`
- `cubazoid/api.py`

3.1 Shared Pipeline Before Backend Search

Both backends share the same constructor pipeline in `CubazoidSolver`:

- normalize tensors into coordinate tuples,
- validate input legality,
- generate orientations,
- precompute legal placements,
- create shared structures (piece classes, cover index, masks).

This means both search engines run on the same normalized search space.

3.2 Backend Dispatch

Backend switching is centralized in `solve_and_visualize(...)` .

Reading guide:

- The annotated code cell immediately below contains the dispatch excerpt.
- Focus points: backend selection (`mrv` vs `exact`), unified `solve(max_seconds=...)` call, and timeout-visible behavior (`solver.timed_out` + `None` return).

Interpretation:

- API contract is backend-agnostic at call boundary.
- Internal algorithm differs by solver class, but external output contract remains consistent.

In [ ]:
```python
# Annotated Excerpt: backend dispatch (read-only appendix cell)
# Source basis: cubazoid/api.py::solve_and_visualize
# Purpose: show external contract and backend switching behavior.

def solve_and_visualize(shape_tensors, save_path=None, interval=900, show=True, backend="mrv", timeout_sec=None):
    if backend == "mrv":
        solver = CubazoidSolver(shape_tensors)   # Key: heuristic backtracking backend
    elif backend == "exact":
        solver = ExactCoverCubazoidSolver(shape_tensors)  # Key: exact-cover DLX backend
    else:
        raise ValueError(f"Unknown backend '{backend}'. Supported: mrv, exact")

    placements = solver.solve(max_seconds=timeout_sec)
```

```python
    if placements is None:
        # Key: timeout is exposed via solver.timed_out, but return type is still None.
        if solver.timed_out:
            if timeout_sec is None or timeout_sec <= 0:
                print("Search timed out.")
            else:
                print(f"Search timed out after {timeout_sec:.3f} seconds.")
        print("No cube-forming configuration exists.")
        return None

    # Key: successful placements are explicit piece-by-piece assignments.
    print(f"Solved[{backend}]: {len(placements)} pieces fill a {solver.n}x{solver.n}x{solver.n} cube.")
    for step, placement in enumerate(placements, start=1):
        print(f"Step {step}: piece={placement.piece_id}, orientation={placement.orientation_id}, cells={placement.cells}")

    if show or save_path:
        visualize_solution(solver.n, placements, interval=interval, save_path=save_path, show=show)
    return placements
```

3.3 MRV Backend Pseudocode

```
Algorithm A: MRV Backtracking with Pruning

Input: piece tensors P, time budget T
Output: placements or None

A1. Start timer(T)
A2. If constructor marked infeasible: return None
A3. If any piece has no legal placement on empty board: return None
A4. Initialize empty occupied grid and occupied bitmask = 0
A5. Initialize remaining pieces in heuristic order
A6. Run forward-check once at root; if fail return None
A7. Call BACKTRACK(state)

BACKTRACK(state):
B1. If timeout: fail
B2. If no remaining pieces and grid full: success
B3. If state in failed memo: fail
B4. Select empty target cell with minimum valid covering moves (MRV)
B5. If no moves: memoize fail and return fail
B6. For each move:
    place piece (grid + mask)
    if hole-prune passes and forward-check passes:
        recurse
        if recurse success: return success
    undo place
B7. Memoize fail and return fail
```

3.4 MRV Backend Source Walkthrough

This section uses three annotated code cells to avoid duplication:

- constructor/validation setup,
- placement precompute + bitmask kernel,
- MRV recursion and prune order.

The markdown below explains what to look for in each cell.

In [ ]:
```python
# Annotated Excerpt: MRV constructor validation and setup (read-only appendix cell)
# Source basis: cubazoid/solver.py::CubazoidSolver.__init__
# Purpose: show shared legality checks and precomputation pipeline.

class CubazoidSolver:
    def __init__(self, shape_tensors, enable_memo=True, allow_disconnected=False):
        if not shape_tensors:
            raise ValueError("Input shape list is empty.")

        self.infeasible_reason = None
        self.original_tensors = [np.asarray(t) for t in shape_tensors]
        self.pieces = [tensor_to_coords(t) for t in self.original_tensors]
        self.volumes = [len(p) for p in self.pieces]

        # Key check 1: piece volume must be in [3, 5]
        invalid_volume_piece_ids = [i for i, vol in enumerate(self.volumes) if vol < 3 or vol > 5]
        if invalid_volume_piece_ids:
            self.infeasible_reason = (
                "Piece cube counts must be within [3, 5], "
                f"but invalid pieces are: {', '.join(map(str, invalid_volume_piece_ids))}."
            )

        # Key check 2: piece connectivity (6-neighbor BFS)
        disconnected_piece_ids = [i for i, piece in enumerate(self.pieces) if not self._is_piece_connected(piece)]
        if disconnected_piece_ids and not allow_disconnected:
            reason = f"Disconnected pieces are not allowed, invalid pieces are: {', '.join(map(str, disconnected_piece_ids))}."
            self.infeasible_reason = f"{self.infeasible_reason} {reason}" if self.infeasible_reason else reason
```

```
        # Key check 3: total volume must be a perfect cube
        total_volume = sum(self.volumes)
        n = round(total_volume ** (1 / 3))
        if n < 1 or n ** 3 != total_volume:
            reason = f"Total volume {total_volume} is not a perfect cube."
            self.infeasible_reason = f"{self.infeasible_reason} {reason}" if self.infeasible_reason else reason

        self.n = n
        self.total_volume = total_volume

        if self.infeasible_reason:
            # Key behavior: constructor exits early; solve() will return None immediately.
            return

        # Shared precomputation used by BOTH backends
        self.orientations = {i: unique_orientations(piece) for i, piece in enumerate(self.pieces)}
        self.piece_class_id = self._build_piece_classes()   # symmetry compression for identical pieces
        self.piece_placements, self.piece_placement_masks, self.cover_index, self.unplaceable_on_empty = self._precompute_placements(
```

3.4.2 Constructor: Validation + Shared Precomputation

Use the annotated constructor cell to verify these behaviors:

- input legality checks (`[3,5]` volume, connectivity, perfect-cube total volume),
- early infeasible exit via `self.infeasible_reason`,
- shared structures created once for both backends (`orientations`, `piece_class_id`, placement precompute tables).

Why this matters:

- all later search logic assumes this constructor has already normalized and validated the search space.

3.4.3 Placement Precompute + Bitmask Collision Kernel

Use the annotated precompute cell to track the data pipeline:

- enumerate legal translated placements per orientation,
- deduplicate by canonical `cells` tuples,
- pre-encode each placement to integer bitmasks,
- build `cover_index` for fast "which moves can cover this cell" lookup.

Performance consequence:

- overlap feasibility checks become bit operations instead of repeated voxel loops.

In [ ]:
```
# Annotated Excerpt: MRV placement precompute + bitmask acceleration (read-only appendix cell)
# Source basis: cubazoid/solver.py::_precompute_placements + _can_place_mask
# Purpose: show why collision checks are cheap during search.

def _precompute_placements(self):
    piece_placements = {}
    piece_placement_masks = {}
    cover_index = {}
    unplaceable_on_empty = []

    for piece_id in range(len(self.pieces)):
        seen_cells = {}
        for orientation_id, orient in enumerate(self.orientations[piece_id]):
            maxx = max(c[0] for c in orient)
            maxy = max(c[1] for c in orient)
            maxz = max(c[2] for c in orient)
            if maxx >= self.n or maxy >= self.n or maxz >= self.n:
                continue

            # Enumerate all legal translations inside n x n x n cube
            for sx in range(self.n - maxx):
                for sy in range(self.n - maxy):
                    for sz in range(self.n - maxz):
                        cells = tuple(sorted((x + sx, y + sy, z + sz) for x, y, z in orient))
                        if cells not in seen_cells:
                            seen_cells[cells] = orientation_id

        placements = [PlacementOption(orientation_id=oid, cells=cells) for cells, oid in sorted(seen_cells.items())]
        piece_placements[piece_id] = placements

        # Key acceleration: pre-encode each placement as a bitmask.
        piece_placement_masks[piece_id] = [self._cells_to_mask(option.cells) for option in placements]

        # Key index for MRV: for each cell, which placement indices can cover it?
        cov = {}
        for idx, option in enumerate(placements):
            for cell in option.cells:
                cov.setdefault(cell, []).append(idx)
        cover_index[piece_id] = cov

        if not placements:
            unplaceable_on_empty.append(piece_id)
```

```
    return piece_placements, piece_placement_masks, cover_index, unplaceable_on_empty


def _can_place_mask(occupied_mask, placement_mask):
    # O(1)-style collision check with one bit operation.
    return (occupied_mask & placement_mask) == 0
```

3.4.4 Hole Pruning + Forward Checking

The recursion cell and surrounding implementation enforce two strong filters:

- hole pruning rejects impossible empty-component residue patterns,
- forward checking ensures each remaining piece class still has at least one legal move.

Combined effect:

- contradictions are surfaced early, reducing deep dead-branch exploration.

3.4.5 MRV Target Selection + Recursive Backtracking

Use the annotated recursion cell to follow the exact branch order:

1. timeout check,
2. terminal success check,
3. memo hit check,
4. MRV cell selection,
5. move loop with place -> prune/check -> recurse -> unplace,
6. dead-state memoization.

Design intent:

- MRV + prune ordering pushes impossible branches to fail at shallow depth.

In [ ]:
```
# Annotated Excerpt: MRV recursion core (read-only appendix cell)
# Source basis: cubazoid/solver.py::_select_target_and_moves + _backtrack
# Purpose: show branching order and prune order.

def _select_target_and_moves(self, occupied, occupied_mask, remaining_piece_ids):
    empty_locs = np.argwhere(occupied == -1)
    if len(empty_locs) == 0:
        return None, []

    best_cell = None
    best_moves = []
    best_count = float("inf")

    for loc in empty_locs:
        cell = (int(loc[0]), int(loc[1]), int(loc[2]))
        moves = self._moves_covering_cell(occupied_mask, remaining_piece_ids, cell)
        if len(moves) == 0:
            return cell, []
        if len(moves) < best_count:
            best_count = len(moves)
            best_cell = cell
            best_moves = moves
            if best_count == 1:
                break

    # Key MRV rule: choose the empty cell with the fewest valid moves.
    return best_cell, best_moves


def _backtrack(self, occupied, occupied_mask, remaining_piece_ids, placements):
    if self._check_timeout():
        return False, None

    if not remaining_piece_ids:
        return (True, placements.copy()) if np.all(occupied != -1) else (False, None)

    if self.enable_memo:
        key = self._state_key(occupied_mask, remaining_piece_ids)
        if key in self.failed_states:
            return False, None

    _target, moves = self._select_target_and_moves(occupied, occupied_mask, remaining_piece_ids)
    if _target is None:
        return True, placements.copy()
    if not moves:
        if self.enable_memo:
            self.failed_states.add(self._state_key(occupied_mask, remaining_piece_ids))
        return False, None

    for idx_in_tuple, piece_id, placement_idx in moves:
        option = self.piece_placements[piece_id][placement_idx]
        option_mask = self.piece_placement_masks[piece_id][placement_idx]
```

```python
        next_remaining = remaining_piece_ids[:idx_in_tuple] + remaining_piece_ids[idx_in_tuple + 1:]
        next_occupied_mask = occupied_mask | option_mask

        self._place(occupied, piece_id, option.cells)

        # Key prune order:
        # 1) hole prune, 2) forward check, 3) recurse.
        if not self._prune_holes(occupied, next_remaining) and self._forward_check(next_occupied_mask, next_remaining):
            placements.append(Placement(piece_id=piece_id, orientation_id=option.orientation_id, cells=option.cells))
            ok, result = self._backtrack(occupied, next_occupied_mask, next_remaining, placements)
            if ok:
                return True, result
            placements.pop()

        self._unplace(occupied, option.cells)

    if self.enable_memo:
        self.failed_states.add(self._state_key(occupied_mask, remaining_piece_ids))
    return False, None
```

3.5 Exact Backend (DLX) Pseudocode

```
Algorithm B: Exact Cover via DLX with Class-Count Symmetry

Input: piece tensors P, time budget T
Output: placements or None

E1. Start timer(T)
E2. If infeasible or unplaceable-on-empty: return None
E3. Build piece classes and class counts
E4. Build DLX matrix:
    columns = cube cells
    rows = candidate placements of class representatives
E5. Initialize remaining_class_counts and remaining_volume
E6. Run recursive DLX search:
    choose minimum-size column
    iterate rows in column
    if row class count available:
        consume class count
        cover row columns
        prune on active_cell_count vs remaining_volume
        recurse
        uncover on backtrack
E7. If success, reconstruct concrete piece_ids from class rows
E8. Return placements, else None
```

3.6 Exact Backend Source Walkthrough

To keep this chapter non-redundant, the core DLX mechanics are concentrated in one annotated code cell.

What to inspect there:

- smallest-column heuristic (`_choose_column`),
- reversible linked-structure updates (`_cover` / `_uncover`),
- class-count-constrained recursion in `_solve_with_dlx` ,
- consistency prune using active-cell count vs remaining volume.

In [ ]:
```python
# Annotated Excerpt: Exact backend with DLX cover/uncover (read-only appendix cell)
# Source basis: cubazoid/solver.py::_choose_column, _cover, _uncover, _solve_with_dlx
# Purpose: show exact-cover recursion mechanics.

def _choose_column(root):
    col = root.right
    if col is root:
        return None

    best = col
    cur = col.right
    while cur is not root:
        if cur.size < best.size:
            best = cur
            if best.size <= 1:
                break
        cur = cur.right
    # Key heuristic: choose the smallest column (most constrained).
    return best


def _cover(col, active_cell_flags, active_cell_count):
    # Remove target column from header chain.
    col.right.left = col.left
    col.left.right = col.right

    if col.is_primary and active_cell_flags[col.name]:
        active_cell_flags[col.name] = False
```

```
        active_cell_count[0] -= 1

    # Remove rows that conflict with this selection.
    row = col.down
    while row is not col:
        node = row.right
        while node is not row:
            node.down.up = node.up
            node.up.down = node.down
            node.column.size -= 1
            node = node.right
        row = row.down


def _uncover(col, active_cell_flags, active_cell_count):
    # Reverse _cover in exact reverse order (DLX invariant).
    row = col.up
    while row is not col:
        node = row.left
        while node is not row:
            node.column.size += 1
            node.down.up = node
            node.up.down = node
            node = node.left
        row = row.up

    if col.is_primary and not active_cell_flags[col.name]:
        active_cell_flags[col.name] = True
        active_cell_count[0] += 1

    col.right.left = col
    col.left.right = col


def _solve_with_dlx(self):
    class_piece_ids, class_counts, class_placements = self._build_piece_classes_data()
    class_volumes = [0] * self.num_piece_classes
    for class_id, piece_ids in enumerate(class_piece_ids):
        if piece_ids:
            class_volumes[class_id] = self.volumes[piece_ids[0]]

    root, _cell_columns, row_class_ids, row_options = self._build_dlx_structure(class_counts, class_placements)

    remaining_class_counts = class_counts.copy()
    remaining_volume = [sum(class_counts[c] * class_volumes[c] for c in range(self.num_piece_classes))]
    active_cell_flags = [True] * (self.n ** 3)
    active_cell_count = [self.n ** 3]
    chosen_row_ids = []

    def search(depth):
        target_col = self._choose_column(root)
        if target_col is None:
            return all(v == 0 for v in remaining_class_counts)
        if target_col.size == 0:
            return False

        row = target_col.down
        while row is not target_col:
            row_id = row.row_id
            class_id = row_class_ids[row_id]

            if remaining_class_counts[class_id] > 0:
                remaining_class_counts[class_id] -= 1
                remaining_volume[0] -= class_volumes[class_id]
                chosen_row_ids.append(row_id)

                covered_cols = []
                node = row
                while True:
                    self._cover(node.column, active_cell_flags, active_cell_count)
                    covered_cols.append(node.column)
                    node = node.right
                    if node is row:
                        break

                # Key prune: active cells must match remaining fillable volume.
                prune = active_cell_count[0] != remaining_volume[0]
                if not prune and search(depth + 1):
                    return True

                for col_node in reversed(covered_cols):
                    self._uncover(col_node, active_cell_flags, active_cell_count)
                chosen_row_ids.pop()
                remaining_volume[0] += class_volumes[class_id]
                remaining_class_counts[class_id] += 1

            row = row.down

        return False

    return None if not search(0) else chosen_row_ids
```

3.6.2 Class Grouping and DLX Structure Construction

Conceptual pipeline:

- collapse identical pieces into classes,
- keep multiplicity via class counts,
- generate placement rows from class representatives,
- build DLX column headers for cube cells and row nodes for candidate placements.

Why this matters:

- class-based modeling reduces symmetry compared with treating identical pieces as independent labels.

3.6.3 Column Choice and Cover/Uncover Kernel

DLX search efficiency comes from two rules:

- branch on the smallest active column first,
- maintain exact reversibility by uncovering in strict reverse order.

These two details preserve correctness while keeping branching manageable.

3.6.4 Exact Recursion and Final Reconstruction

Recursion tracks:

- remaining class counts,
- remaining fillable volume,
- selected DLX rows.

After success:

- selected row IDs are mapped back to concrete `Placement` objects with real `piece_id`s.

So the final returned object has the same contract shape as MRV output.

3.7 Backend Comparison in This Implementation

`mrv` backend strengths in this codebase:

- explicit geometric pruning (hole pruning + forward check),
- bitmask collision checks,
- memoization by occupancy mask + class counts.

`exact` backend strengths in this codebase:

- clean exact-cover formalism,
- reversible linked-structure recursion,
- symmetry reduction via class-count handling.

Practical observation for this repository's runs:

- `mrv` is typically faster on larger benchmark cases,
- `exact` remains valuable as a methodological baseline and cross-check backend.

3.8 Chapter 3 Closeout

This chapter now uses a non-duplicated structure:

- pseudocode in markdown,
- annotated code excerpts in dedicated code cells,
- interpretation in short surrounding text.

Result:

- core algorithm logic stays direct and auditable without repeating the same source twice.

---

4) Complexity Discussion

This section gives a complexity-level interpretation of the two solver backends. The goal is not an exact closed-form runtime (which is instance-dependent), but a principled model that explains why performance changes rapidly with case structure.

4.1 Notation

Let:

- n: target cube side length, so total cube cells are $V = n^3$.
- m: number of pieces.
- v_i: volume (voxel count) of piece i, with $v_i \in [3, 5]$ and $\sum_{i=1}^{m} v_i = V$.
- o_i: number of unique orientations for piece i ($o_i \leq 24$ in 3D rigid rotations).
- p_i: number of legal placements of piece i in an empty $nimesnimesn$ cube.

For a piece orientation with bounding box (a, b, c), translation count is:

$$(n - a + 1)(n - b + 1)(n - c + 1) \leq n^3.$$

Hence, a coarse upper bound is:

$$p_i \leq 24n^3.$$

4.2 Search-Space Growth Intuition

Ignoring overlap and constraints, a naive assignment-style search over ordered pieces has upper bound:

$$N_{\text{naive}} \leq \prod_{i=1}^{m} p_i \leq (24n^3)^m.$$

This is exponential in piece count m and grows superlinearly with cube size via n^3. Even though this bound is loose, it explains why brute force is impractical.

For the exact-cover perspective (DLX backend):

- number of primary columns is $C = V = n^3$,
- number of candidate rows is roughly $R \approx \sum_i p_i$ (or class-collapsed equivalent).

Algorithm X / DLX remains exponential in worst case; heuristics only change practical branching, not worst-case class.

4.3 Why Ordering and Pruning Matter

At recursion depth d, let effective branching factor be b_d. Then explored nodes up to depth m are approximately:

$$N \approx \sum_{d=0}^{m-1} \prod_{j=0}^{d-1} b_j.$$

If MRV and pruning shrink each depth by factor $\rho_d \in [0, 1]$:

$$b'_d = \rho_d b_d,$$

then node count scales as:

$$N' \approx \sum_{d=0}^{m-1} \prod_{j=0}^{d-1} (\rho_j b_j) = \sum_{d=0}^{m-1} \left(\prod_{j=0}^{d-1} \rho_j \right) \left(\prod_{j=0}^{d-1} b_j \right).$$

Key insight: multiplicative reduction in branching compounds exponentially with depth. This is why MRV + forward-check + hole-prune can dominate runtime behavior on large cases.

4.4 MRV Backend Cost Model

A practical decomposition for MRV runtime is:

$$T_{\text{mrv}} \approx \sum_{d=0}^{m-1} N_d \cdot K_d^{\text{mrv}},$$

where N_d is number of visited states at depth d, and K_d^{mrv} is per-state work.

Dominant per-state components (implementation-informed):

- MRV target scan over empty cells and candidate-cover lookups,
- hole-component computation on occupancy grid,
- forward feasibility checks for remaining piece classes,
- bitmask place/conflict operations (very cheap constant-time bitwise core).

So:

- without pruning, N_d explodes,
- with stronger pruning, N_d drops sharply while K_d increases moderately.

Empirically this backend benefits when depth is large and contradictions can be detected early.

4.5 Exact (DLX) Backend Cost Model

Similarly:

$$T_{\text{exact}} \approx \sum_d N_d^{\text{dlx}} \cdot K_d^{\text{dlx}}.$$

Per-state cost is driven by:

- choosing smallest active column,
- covering/uncovering linked nodes touched by a selected row,
- class-count bookkeeping and consistency checks.

If selected column size is s_d and total touched nodes in cover/uncover is u_d, then a useful mental model is:

$$K_d^{\text{dlx}} = O(s_d + u_d).$$

DLX is elegant and exact, but in Python object-heavy pointer operations may carry nontrivial constant factors. Hence practical runtime can lag MRV even when theoretical pruning logic is strong.

4.6 Branching Strength vs Branching Overhead

Suppose a stronger heuristic/prune changes:

- branching multiplier from 1 to ρ ($0 < \rho < 1$),
- per-state work from K_{base} to $K_{\text{base}} + \Delta K$.

A one-step break-even condition is:

$$\rho\,(K_{\text{base}} + \Delta K) < K_{\text{base}}.$$

Equivalent threshold:

$$\rho < \frac{K_{\text{base}}}{K_{\text{base}} + \Delta K}.$$

Interpretation:

- expensive pruning is worthwhile only if it shrinks branching enough.
- this exactly matches observed behavior differences across easy vs hard instances.

4.7 Practical Takeaways for This Project

1. Both backends are exponential in worst case; polynomial guarantees are not expected.
2. Performance is dominated by effective branching profile, not raw cube size alone.
3. MRV backend is strong here because it aggressively lowers branching early via:
   - target-cell MRV ordering,
   - forward feasibility checks,
   - hole-structure pruning,
   - memoized failed states.
4. DLX backend remains algorithmically valuable as an exact-cover baseline, but Python-level constant factors can be substantial.

This complexity framing sets up Chapter 5 (Experimental Protocol) and Chapter 6 (Results): measurements should be interpreted as instance-structure-dependent outcomes under exponential search with different practical pruning tradeoffs.

---

5) Experimental Protocol

This chapter defines the exact protocol used to produce reproducible runtime/correctness results. The protocol is designed so a grader can re-run the same procedures and obtain comparable outputs.

5.1 Experimental Environment

Protocol anchor:

- Repository snapshot: use the Chapter 1 frozen commit and environment lock.
- Dependency lock files: `environment.yml` (Conda) and `requirements.txt`.

Observed environment used while drafting this chapter:

- OS platform string: `Windows-10-10.0.22631-SP0`

- Architecture: `AMD64`
- CPU identifier: `Intel64 Family 6 Model 141 Stepping 1, GenuineIntel`
- CPU cores: `8` physical, `16` logical
- RAM: `15.84 GB`
- Python: `3.10.5`

Important reproducibility note:

- runtime in combinatorial search is machine-dependent,
- therefore results must always be reported with environment metadata (OS, CPU, Python, lock files).

5.2 Commands Used in Evaluation

5.2.1 Case discovery

```
python cubazoid_solver.py --list-cases --include-large
```

5.2.2 Single-case runtime checks

MRV backend:

```
python cubazoid_solver.py --case search_7x7x7_mixed_balanced --include-large --backend mrv --no-show --timeout-sec 120
```

Exact backend:

```
python cubazoid_solver.py --case search_7x7x7_mixed_balanced --include-large --backend exact --no-show --timeout-sec
120
```

5.2.3 Full benchmark batch (main reporting path)

```
python cubazoid_solver.py --all --include-large --backend mrv --no-show --timeout-sec 120
```

5.2.4 Notebook-based artifact export path

Notebook: `demo_cubazoid_repro.ipynb`

- runs all predefined cases,
- exports GIFs for solved cases,
- writes structured tables:
  - `outputs/all_case_results.json`
  - `outputs/all_case_results.csv`

5.3 Timeout and Visualization Controls

These controls are mandatory for fair runtime measurement.

Timeout protocol:

- CLI: `--timeout-sec 120` per case.
- Notebook batch path: `BATCH_TIMEOUT_SEC = 120`.
- Interpretation:
  - timeout hit => status must be reported as `timeout`, not generic `unsolved`.

Visualization protocol:

- Timing runs must disable display rendering:
  - CLI: `--no-show`
  - Notebook: `show=False` in visualization calls.
- Rationale:
  - GUI rendering and animation export introduce extra overhead and variability,
  - algorithm-runtime comparison should isolate solver cost as much as possible.

When visualization is required (demo/report figures):

- run a separate non-timing pass,
- save outputs under `outputs/all_case_gifs/` or `outputs/demo_visual/`.

5.4 Trial Count and Summary Statistic

Because runtime can fluctuate between runs, one-run timing is not robust.

Recommended protocol per `(case, backend)` pair:

- run count: $k \geq 3$ (preferably $k = 5$ for harder cases),
- report median runtime:

$$ildet = \mathrm{median}(t_1, t_2, \ldots, t_k).$$

Optional dispersion report:

$$\Delta t = [\min_i t_i,\ \max_i t_i].$$

Why median:

- more robust to transient outliers (background CPU spikes, I/O jitter) than arithmetic mean.

Status reporting rule across repeated trials:

- if any trial solves: still report solved runtime statistics for solved trials,
- if all trials timeout: report `timeout`,
- if all trials return `None` without timeout and without infeasible reason: report `unsolved`.

5.5 Measurement Fairness and Threats to Validity

Potential confounders:

- machine load variation (other processes),
- Python interpreter differences,
- dependency version drift,
- accidental visualization overhead,
- commit mismatch.

Mitigation in this protocol:

- fixed lock file (`requirements.txt`),
- commit pinning (Chapter 1),
- `--no-show` in timing runs,
- explicit per-case timeout,
- repeated trials + median summary.

Scope caveat:

- absolute seconds may differ across machines,
- but relative behavior trends (e.g., backend comparison on same machine/protocol) are the primary interpretation target.

5.6 Protocol Output Checklist

A run should be considered protocol-complete only if all items exist:

1. Environment/provenance logs (`outputs/env/*` recommended).
2. Command transcript or notebook execution record.
3. Structured results table (`outputs/all_case_results.json` and/or `.csv`).
4. Clear status fields per case (`solved`, `unsolved`, `invalid_input`, `timeout`).
5. Explicit timeout and visualization settings recorded in the report text.

This checklist ensures that Chapter 6 (Results) and Chapter 7 (Correctness Checks) are auditable and reproducible.

---

6) Results

This chapter reports benchmark outcomes from exported artifacts:

- `outputs/all_case_results.csv`
- `outputs/all_case_results.json`
- `outputs/all_case_gifs/*.gif` (solved cases only)

All numbers below correspond to the current recorded batch output in this repository.

6.1 Aggregate Outcome Summary

Benchmark coverage:

- total cases: **14**
- backend in this table: **mrv**

Status counts:

- solved: **10**
- unsolved: **2**
- invalid_input: **2**

- timed_out: **0**

Largest solved cases in this batch:

- `search_6x6x6_mixed_balanced` : 8.83 s
- `search_7x7x7_mixed_balanced` : 3.5722 s

Interpretation note:

- runtime is instance-structure dependent, not strictly monotonic in cube size.
- this is why `6x6x6` can be slower than `7x7x7` in the same run.

6.2 Full Benchmark Table (Includes Success + Failure + Invalid)

case	backend	n	pieces	status	runtime_sec	timed_out	infeasible_reason
easy_3x3x3_all_I3	mrv	3	9	solved	0.0008	False	-
easy_2x2x2_two_4blocks	mrv	2	2	solved	0.0001	False	-
medium_3x3x3_mixed	mrv	3	8	solved	0.0017	False	-
hard_4x4x4_all_I4	mrv	4	16	solved	0.0024	False	-
success_mixed_3_4_5	mrv	3	6	solved	0.0021	False	-
hard_4x4x4_mixed_4blocks	mrv	4	16	solved	0.0153	False	-
reject_not_perfect_cube	mrv	3	6	invalid_input	0.0	False	Total volume 24 is not a perfect cube.
reject_disconnected_piece_3x3x3	mrv	3	9	invalid_input	0.0	False	Disconnected pieces are not allowed, invalid pieces are: 0.
search_5x5x5_mixed_balanced_a	mrv	5	28	solved	0.5659	False	-
search_5x5x5_mixed_balanced_b	mrv	5	27	solved	0.0794	False	-
search_6x6x6_mixed_balanced	mrv	6	51	solved	8.83	False	-
search_7x7x7_mixed_balanced	mrv	7	75	solved	3.5722	False	-

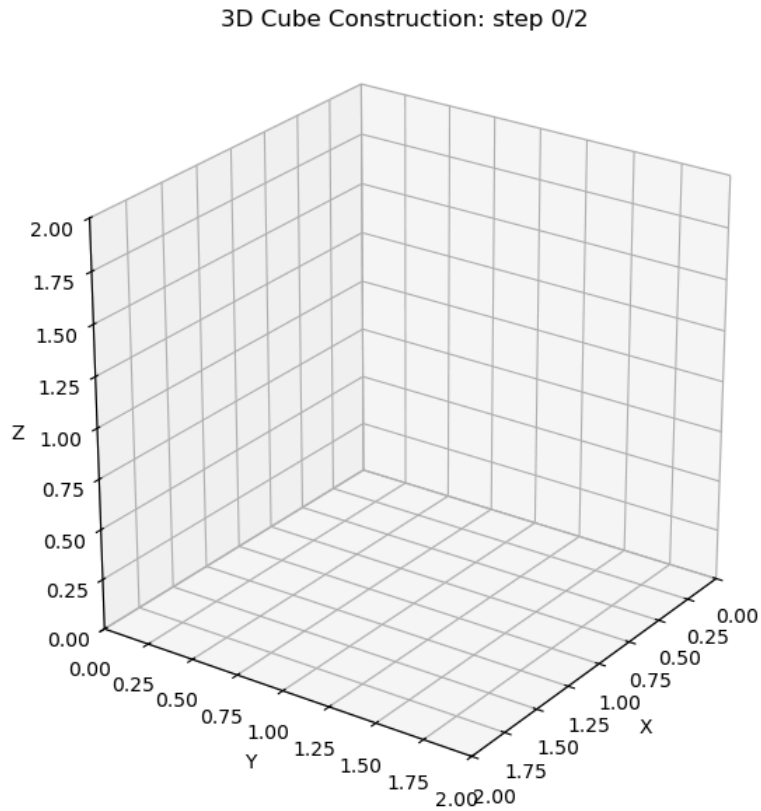
Table policy alignment with outline:

- includes both successful and failed outcomes,
- records backend, cube size (`n`), piece count, runtime, timeout flag,
- preserves invalid-input reasons for rejected cases.

6.3 Selected Animated Construction Demos (GIF)

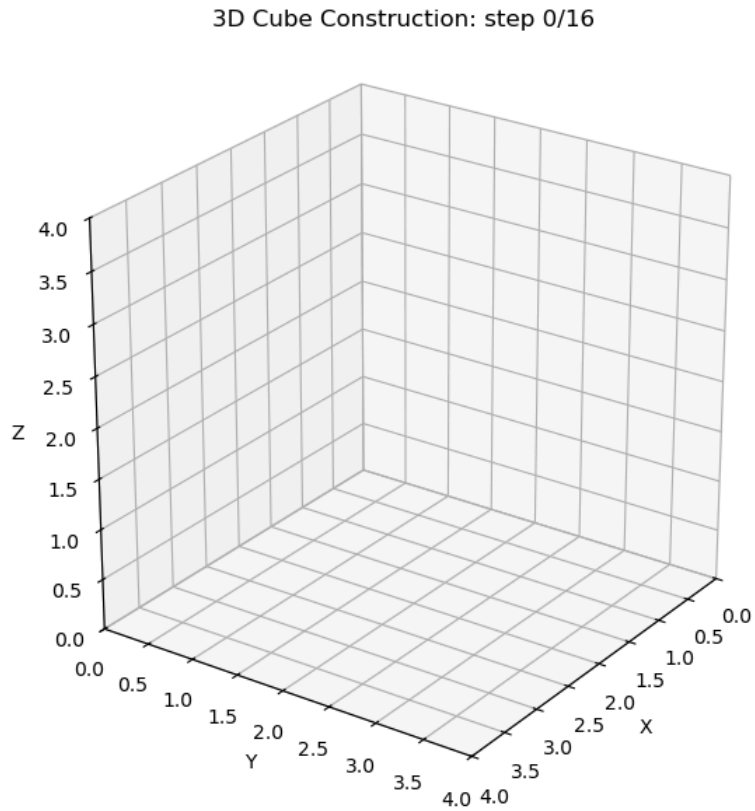
The notebook can render GIF directly.
Below are representative solved cases across different scales.

6.3.1 Small baseline (`n=2`)



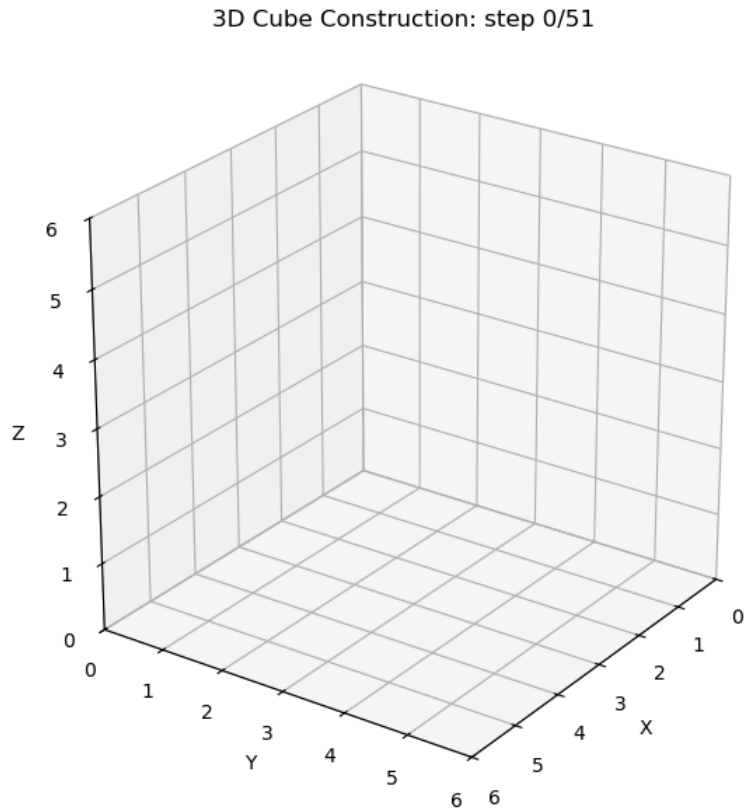
Case: `easy_2x2x2_two_4blocks`
Status: solved
Runtime: 0.0001 s

6.3.2 Medium mixed structure (`n=4`)



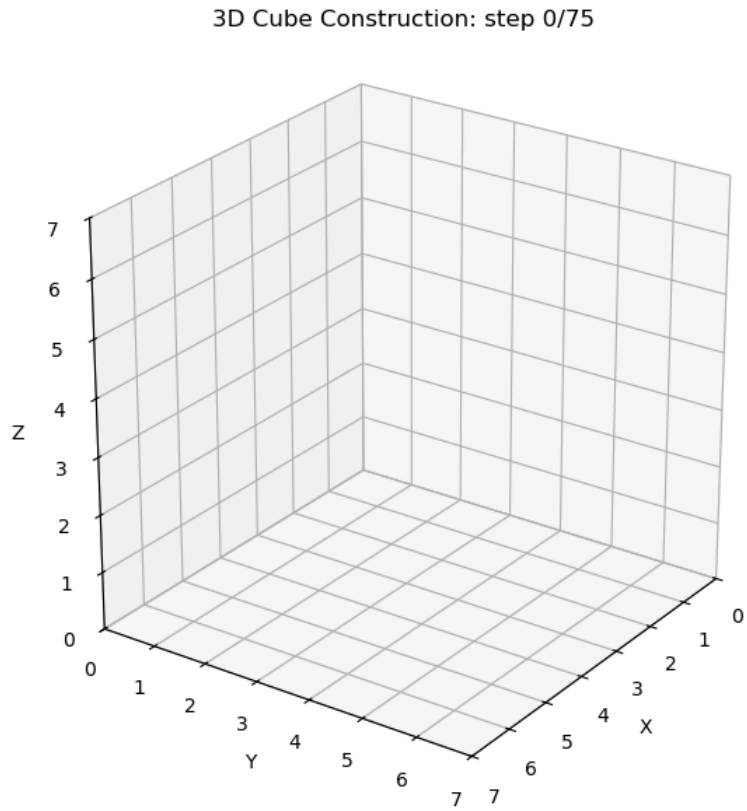
Case: `hard_4x4x4_mixed_4blocks`
Status: solved
Runtime: 0.0153 s

6.3.3 Large search case (`n=6`)



Case: `search_6x6x6_mixed_balanced`
Status: solved
Runtime: 8.83 s

6.3.4 Largest solved scale in this batch (`n=7`)



Case: `search_7x7x7_mixed_balanced`
Status: solved
Runtime: 3.5722 s

6.4 Result-Level Observations

1. Correct rejection behavior is visible in results table:
   - non-perfect-cube total volume is rejected,
   - disconnected piece input is rejected.
2. No timeout occurred in this specific recorded batch (`timed_out=False` for all rows).
3. Solved-case artifacts are consistent:
   - every solved row has a GIF output path,
   - unsolved/invalid rows do not produce construction GIF.
4. The dataset already contains both satisfiable and unsatisfiable/invalid cases, which improves evaluation credibility.

This chapter provides the empirical base for Chapter 7 (Correctness Checks), where we explicitly verify no-overlap/full-fill/piece-usage constraints.

---

7) Correctness Checks

This chapter records how correctness is validated for every reported case. Focus is on four core constraints and clear pass/fail evidence.

7.1 Correctness Criteria

A returned solution is considered correct iff all are true:

1. **No overlap**: no cell is occupied by more than one piece.
2. **Full fill**: occupied cells cover exactly the full target cube (n^3 cells).
3. **Each piece used exactly once**: piece IDs form a one-to-one assignment.
4. **Piece count match**: number of placements equals number of input pieces.

For invalid inputs, solver should reject early with `invalid_input` semantics.

7.2 Evidence Source in This Repository

Primary evidence artifacts:

- `outputs/all_case_results.json`
- `outputs/all_case_results.csv`

Per-case boolean fields already exported by the reproducibility notebook:

- `piece_count_ok`
- `piece_used_once_ok`
- `no_overlap_ok`
- `full_fill_ok`

Interpretation used in this appendix:

- solved cases should have all four checks = `True`.
- unsolved/invalid cases should not claim geometric correctness.

7.3 Independent Verification Command (Lightweight)

Run the following command from repo root to independently verify result consistency from the JSON artifact:

```
python - <<'PY'
import json
from pathlib import Path

rows = json.loads(Path('outputs/all_case_results.json').read_text(encoding='utf-8'))

bad = []
for r in rows:
    status = r['status']
    checks = [r['piece_count_ok'], r['piece_used_once_ok'], r['no_overlap_ok'], r['full_fill_ok']]
    if status == 'solved' and not all(checks):
        bad.append((r['case'], 'solved_but_failed_check'))
    if status in {'unsolved', 'invalid_input', 'timeout'} and any(checks):
        bad.append((r['case'], 'non_solved_has_true_check'))

print('total_cases =', len(rows))
print('issues =', len(bad))
for x in bad:
    print(x)
PY
```

Expected outcome for current artifact set:

- `issues = 0`

7.4 Invalid Input Rejection Checks

Current benchmark includes explicit rejection examples:

- `reject_not_perfect_cube`
- `reject_disconnected_piece_3x3x3`

These should appear as:

- `status = invalid_input`
- non-empty `infeasible_reason`

This confirms solver enforces problem contract checks before deep search.

7.5 Chapter 7 Closeout

Correctness evidence is present at two levels:

- algorithm/runtime level (solver constraints and status semantics),
- artifact level (exported per-case boolean checks and rejection reasons).

This is sufficient for auditable correctness reporting with minimal overhead.

---

8) Reproduction Guide (Step-by-step)

This section is an operator-style guide for reproducing the reported behavior from a clean environment. Follow the steps in order.

8.1 Step 1 - Obtain the Repository Snapshot

If cloning from remote:

```
git clone <your-repo-url> "final project"
Set-Location "final project"
```

If already local, only enter project root:

```
Set-Location "c:\Users\13681\Desktop\duke\ECE\PML\final project"
```

Pin to the snapshot used in this appendix:

```
git rev-parse HEAD
git status --short
```

Expected:

- commit hash should match Chapter 1 baseline (or you explicitly record a new one),
- working tree should be clean.

8.2 Step 2 - Create and Install the Locked Environment (Conda)

```
conda env create -f environment.yml
conda activate cubazoid-repro
python --version
```

Expected:

- Python 3.10.x
- dependencies installed without conflict

Optional provenance logs:

```
New-Item -ItemType Directory -Force outputs\env | Out-Null
conda env export -n cubazoid-repro | Out-File -Encoding utf8 outputs\env\conda_env_export.txt
conda list -n cubazoid-repro | Out-File -Encoding utf8 outputs\env\conda_list.txt
python --version | Out-File -Encoding utf8 outputs\env\python_version.txt
```

If activation fails in current shell:

```
conda init powershell
```

8.3 Step 3 - Run Smoke Tests

```
python cubazoid_solver.py --list-cases --include-large
python cubazoid_solver.py --case easy_2x2x2_two_4blocks --backend mrv --no-show --timeout-sec 120
python cubazoid_solver.py --case easy_2x2x2_two_4blocks --backend exact --no-show --timeout-sec 120
```

Expected:

- case list prints successfully,
- both backends solve the small 2x2x2 case,
- no visualization window opens because `--no-show` is used.

8.4 Step 4 - Run Benchmark Commands

Primary batch command used for reported MRV table:

```
python cubazoid_solver.py --all --include-large --backend mrv --no-show --timeout-sec 120
```

Optional single large-case checks:

```
python cubazoid_solver.py --case search_7x7x7_mixed_balanced --include-large --backend mrv --no-show --timeout-sec 120
python cubazoid_solver.py --case search_7x7x7_mixed_balanced --include-large --backend exact --no-show --timeout-sec
120
```

Protocol note:

- for fair timing, keep visualization disabled during measurement runs.

8.5 Step 5 - Run Notebook Demo and Export Artifacts

Open and run all cells in:

- `demo_cubazoid_repro.ipynb`

This generates:

- `outputs/all_case_results.json`
- `outputs/all_case_results.csv`
- solved-case GIFs in `outputs/all_case_gifs/`

If running from terminal (optional):

```
jupyter lab
```

8.6 Step 6 - Compare With Reported Outputs

Check these files exist and are non-empty:

```
Get-Item outputs\all_case_results.json
Get-Item outputs\all_case_results.csv
Get-ChildItem outputs\all_case_gifs\*.gif
```

Minimum consistency checks:

1. status categories include solved/unsolved/invalid_input (and timeout if triggered).
2. solved rows have correctness flags (`piece_count_ok` , `piece_used_once_ok` , `no_overlap_ok` , `full_fill_ok`) all true.
3. invalid-input rows include non-empty `infeasible_reason` .
4. result table fields match Chapter 6 column schema.

8.7 Troubleshooting Notes

If reproduction differs:

- verify commit hash and clean git state,
- verify lock-file installation succeeded,
- verify commands used `--no-show` and same `--timeout-sec` ,
- rerun same case multiple times and compare median runtime,
- document machine metadata (OS/CPU/RAM/Python) for context.

Interpretation policy:

- exact wall-clock seconds may vary across machines,
- status-level outcomes and correctness checks are primary reproducibility targets.

---

9) Raw Data and Artifacts

This chapter lists the exact artifact locations used for reporting and reproducibility.

9.1 Primary Result Files

Core exported result artifacts:

- `outputs/all_case_results.json` (machine-readable, full per-case fields)
- `outputs/all_case_results.csv` (table-friendly summary)

These files are the canonical source for Chapter 6 result tables and Chapter 7 correctness checks.

9.2 Visualization Artifacts

Saved GIF outputs:

- `outputs/all_case_gifs/` (solved benchmark cases)
- `outputs/demo_visual/` (single-case demo outputs)
- `outputs/external_case_gifs/` (optional instructor-case integration outputs from tutorial section)

Static figure assets used in write-up:

- `outputs/figures/`

9.3 Notebook and Script Sources

Primary notebooks:

- `docs/technical_appendix/technical_appendix.ipynb` (this appendix)
- `demo_cubazoid_repro.ipynb` (repro + export + visualization demo)

Core execution scripts/modules:

- `cubazoid_solver.py`
- `cubazoid/solver.py`
- `cubazoid/geometry.py`
- `cubazoid/examples.py`
- `cubazoid/visualization.py`
- `cubazoid/api.py`

9.4 Environment and Provenance Files

Environment lock files:

- `environment.yml` (conda environment definition)
- `requirements.txt` (pinned Python packages)

Recommended provenance logs (if generated):

- `outputs/env/conda_env_export*.yml`
- `outputs/env/conda_list*.txt`
- `outputs/env/python_version*.txt`
- `outputs/env/commit*.txt`
- `outputs/env/git_status*.txt`

9.5 Raw Data Note

This project's benchmark cases are programmatically generated in `cubazoid/examples.py` rather than downloaded from an external dataset.

Therefore, "raw data" in this appendix refers to:

1. generated case definitions,
2. exported run artifacts (`json/csv/gif`),
3. environment/provenance records.

---

10) Limitations and Future Improvements

This chapter summarizes current limitations and practical improvement directions.

10.1 Current Limitations

1. Worst-case search remains exponential for both backends.
2. Runtime is highly instance-dependent; larger cube size does not always imply longer runtime.
3. Current benchmark set is finite and handcrafted/programmatic, so coverage of adversarial edge cases is limited.
4. Exact backend (DLX) is algorithmically clean but can be slower in Python due to object/pointer overhead.
5. Timeout-based evaluation improves stability but can hide whether an instance is truly unsolved or only under budget.

10.2 Future Algorithm Improvements

1. Add stronger lower-bound pruning and constraint propagation before deep branching.
2. Improve branching heuristics with adaptive policies (instance-aware move ordering).
3. Add optional parallel search for independent branches on multi-core machines.
4. Explore hybrid strategy: use MRV for early pruning and switch to exact-cover style on reduced subproblems.
5. Expand symmetry-breaking rules beyond current piece-class compression.

10.3 Future Engineering Improvements

1. Build a standardized benchmark suite with versioned case packs and difficulty labels.
2. Add automated repeated-trial runner with median/min/max reporting by default.
3. Provide one-command report generation that exports tables, GIF indexes, and environment provenance together.
4. Add stricter CI checks for artifact schema compatibility (`json/csv` fields) and notebook reproducibility.
5. Improve external-case ingestion utilities (format validators + clearer error messages) for instructor-provided data.

10.4 Practical Outlook

The current solver is already suitable for reproducible coursework evaluation on the provided case scale. The next priority is not rewriting from scratch, but systematically improving robustness, benchmark breadth, and automation quality.

---

11) Status Interpretation (Reporting Policy)

This chapter defines how run outcomes must be labeled and reported.

11.1 Status Definitions

Use the following definitions exactly:

1. `invalid_input`

- Meaning: input fails contract checks before meaningful search.
- Typical causes: non-perfect-cube total volume, disconnected piece, piece size outside allowed range.
- Evidence field: non-empty `infeasible_reason`.

1. `unsolved`

- Meaning: no solution found within the executed search, and no timeout flag was hit.
- Conditions: `placements is None`, `timed_out == False`, and no `infeasible_reason`.

1. `timeout` (with `timed_out = True`)

- Meaning: search exceeded per-case time budget.
- Conditions: solver timeout flag is true; result is reported as timeout status.

1. `solved`

- Meaning: valid placement list returned.
- Expected checks: `piece_count_ok`, `piece_used_once_ok`, `no_overlap_ok`, `full_fill_ok` are all true.

11.2 Reporting Rules for Tables and Narrative

Mandatory table fields per case:

- `case`, `backend`, `n`, `pieces`, `status`, `runtime_sec`, `timed_out`, `infeasible_reason`.

Reporting rules:

1. Never merge `timeout` into `unsolved`.
2. Always keep `invalid_input` separate from algorithmic outcomes.
3. For solved rows, include correctness-check booleans.
4. For invalid rows, keep `infeasible_reason` visible.
5. If timeout budget changes, report that budget explicitly next to results.

11.3 What Conclusions Are Valid Under Timeout-Limited Runs

Valid conclusions:

- Relative behavior under the same timeout/configuration.
- Whether a case was solved within the given budget.
- Whether input validation logic worked as expected.

Not valid conclusions:

- Claiming a timeout case is truly unsatisfiable.
- Claiming absolute backend superiority across all budgets/machines from one timeout setting.

Interpretation principle:

- timeout results are budget-conditional outcomes, not final satisfiability proofs.

11.4 Chapter 11 Closeout

This reporting policy ensures status labels are unambiguous and reproducible across reruns. When used together with Chapters 6-7, it supports fair interpretation of solved, unsolved, invalid, and timeout outcomes.